

Modernizing a critical platform without stopping the business

LEGACY MODERNIZATION · APPLICATION ARCHITECTURE · DATA SYSTEMS

50+

APPLICATION VIEWS CONSOLIDATED

Zero

OPERATIONAL DOWNTIME

10–15 ms

SEARCH-QUERY LATENCY

THE CHALLENGE

Filmpac's internal Studio platform supported creative production, keywording, e-commerce administration, management reporting, and partner workflows. It had also been rebuilt repeatedly, accumulating fragmented processes, aging frontend architecture, and difficult-to-maintain services.

Studio eventually supported more than 50 application views. Another wholesale rewrite would have introduced substantial operational risk, while continuing indefinitely on the existing architecture was increasingly costly.

THE APPROACH

As technical lead, I chose incremental modernization rather than another full replacement.

Using a Strangler-style approach, I separated legacy and modern application paths while both remained in production. A federated GraphQL layer insulated the frontend from existing services, and coordinated Next.js routing allowed features to move gradually from older Pages Router code to newer App Router patterns.

I also redesigned the data architecture around purpose-specific systems: PostgreSQL for relational application and reporting data, DynamoDB for canonical partner documents, and Algolia for high-performance search and e-commerce workflows. Multiple complex migrations were completed without operational downtime.

Customer-facing search and filtering were moved out of an overloaded WordPress/WooCommerce database, removing a bottleneck that contributed to 5–20 second page loads and occasional waits approaching a minute.

THE RESULT

The modernization allowed new functionality to continue shipping while legacy components were retired incrementally.

Studio helped the creative organization process more video clips in six months than in the previous three to four years, and remained sustainable as the product organization contracted from roughly nine people to two engineers.

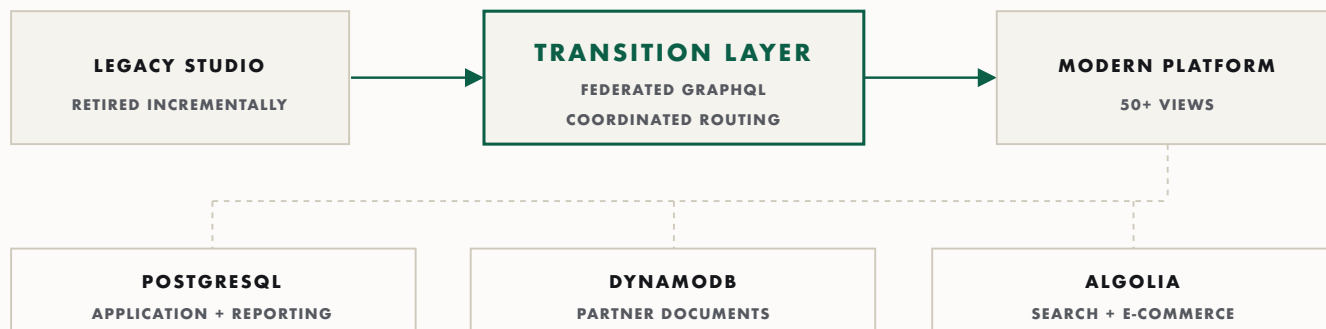
WHY IT MATTERED

Modernization did not require betting the company on another rewrite.

The durable approach was to identify clean boundaries, preserve functioning systems, replace weak components incrementally, and keep production moving throughout.

TECHNOLOGY

TypeScript · React · Next.js · GraphQL · PostgreSQL · DynamoDB · Algolia · AWS



BOTH PATHS LIVE IN PRODUCTION WHILE THE LEGACY SIDE IS RETIRED PIECE BY PIECE